

Dropout, Batch Normalization & Early Stopping

Goal: Improve ANN Accuracy and Generalization

These three techniques are mainly used to solve overfitting and make neural networks train more effectively.

Why do we need these techniques?

Suppose we train an ANN on student data.

The model gets:

- **Training accuracy = 98%**
- **Validation accuracy = 78%**
- **The model has learned the training data too well, including noise and unnecessary patterns.**

This is called: Overfitting

Our goal is not simply to get high training accuracy.

We want the model to perform well on new/unseen data.

That's where:

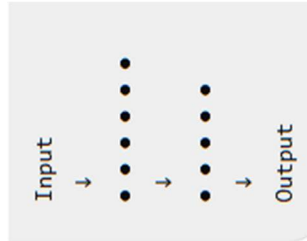
- **Dropout**
- **Batch Normalization**
- **Early Stopping**

become useful.

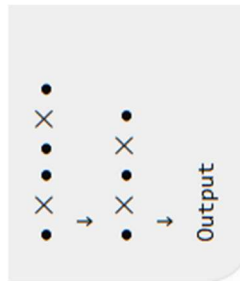
Dropout

Dropout randomly turns off some neurons during training.

For example, suppose our ANN has:



During training, Dropout may randomly deactivate some neurons:



The dropped neurons do not participate in that particular training iteration.

Without dropout, the network may become too dependent on particular neurons.

Dropout forces the network to learn more robust features.

Example

Suppose dropout rate = 0.5.

That means approximately 50% of the neurons are randomly ignored during each training iteration.

Practical Implementation:

```
model.add(Dense(128, activation='relu'))  
model.add(Dropout(0.5))
```

```
model.add(Dense(64, activation='relu'))  
model.add(Dropout(0.3))
```

What does Dropout solve?

Overfitting

It generally causes training accuracy to become a little lower, but can improve validation/test performance.

Batch Normalization

Batch Normalization is usually called: BatchNorm

It normalizes the activations of a neural network during training.

During training, the distribution of values flowing through the network can keep changing.

This can make training:

- slower
- unstable
- harder to optimize

Batch Normalization helps keep these values in a more controlled range.

Original values



Calculate mean & variance



Normalize



Scaled/shifted values

The basic normalization is:

$$z_{norm} = \frac{z - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

Where:

- μ = batch mean
- σ^2 = batch variance
- ϵ = small value to avoid division by zero

Keras Implementation

```
model.add(Dense(128, activation='relu'))
```

```
model.add(BatchNormalization())
```

```
model.add(Dense(64, activation='relu'))
```

```
model.add(BatchNormalization())
```

Benefits of Batch Normalization

It can:

- Make training more stable
 - Often speed up training
 - Allow useful learning rates
 - Improve optimization
 - Sometimes provide a regularizing effect
-

Early Stopping

During training:

Epoch 1 → Validation accuracy = 70%

Epoch 2 → Validation accuracy = 75%

Epoch 3 → Validation accuracy = 80%

Epoch 4 → Validation accuracy = 83%

Epoch 5 → Validation accuracy = 84%

Epoch 6 → Validation accuracy = 83%

Epoch 7 → Validation accuracy = 81%

Training accuracy may still be increasing.

But validation performance has started getting worse.

That indicates: The model is beginning to overfit.

So, we stop training early.

Early stopping practical Implementation

```
from tensorflow.keras.callbacks import EarlyStopping
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

model.fit (
    X_train,
    y_train,
    validation_split=0.2,
    epochs=100,
    callbacks=[early_stop]
)
```

What does patience=5 mean?

If validation loss doesn't improve for 5 consecutive epochs, training stops.

restore_best_weights=True

This is important.

Suppose:

Epoch 10 → best validation performance

Epoch 11

Epoch 12

Epoch 13

Epoch 14

Epoch 15 → training stops

The model goes back to the weights from Epoch 10, where validation performance was best.

Difference between the three

Technique	Main purpose
Dropout	Reduce overfitting
Batch Normalization	Stabilize/accelerate training
Early Stopping	Stop training before overfitting
